

1^a Série

Trabalho realizado por:

Gustavo Costa | N^o 52808

Ian Frunze | N^o 52867

Rafael Pereira | N^o 52880

Turma: **LEIC24D**

Docentes: **Nuno Leite**

Licenciatura em Engenharia Informática e de Computadores

Algoritmos e Estruturas de Dados (AED)

2024 / 2025 - Semestre de Verão

Conteúdo

1	Introdução	2
2	Algoritmos Elementares	3
2.1	Função findMinDifference	3
2.2	Função counter	3
2.3	Função createSortedPartitions	4
3	Análise e Desempenho	5
3.1	Função xpto	5
3.2	Função someSort	8
3.3	Proposta de algoritmo de ordenação	11
3.3.1	Implementação proposta	11
3.3.2	Complexidade Temporal	12
3.3.3	Complexidade Espacial	12

1 Introdução

O presente relatório foca-se na análise e resolução dos problemas propostos nas secções 2 e 3 do enunciado, nomeadamente na análise de desempenho algorítmico e no desenvolvimento de uma solução para ordenação de ficheiros de grande dimensão.

Importa referir que, conforme estipulado no enunciado, as soluções relativas à primeira secção (Algoritmos Elementares) não serão incluídas no corpo deste relatório, sendo antes apresentadas em anexo sob a forma de código implementado. Deste modo, o relatório prossegue diretamente para a análise da Secção 2 - Análise de Desempenho.

Na Secção 2, abordaremos:

- A análise assintótica do algoritmo `xpto`;
- O estudo da complexidade do algoritmo `someSort`;
- A proposta de implementação de um algoritmo de ordenação estável.

Posteriormente, na Secção 3, desenvolveremos e analisaremos uma aplicação para ordenação de ficheiros de dimensão arbitrária, que contempla duas abordagens distintas de implementação, uma delas sem recorrer ao uso das estruturas de dados disponíveis em `kotlin.collections` ou `java.util` e outra recorrendo às mesmas. A avaliação experimental comparativa destas implementações permitirá verificar a eficácia das soluções propostas, com especial atenção à complexidade temporal desejada de $\mathcal{O}(n \cdot \log_2 n)$.

Este relatório não só cumpre os requisitos técnicos do enunciado, mas também proporciona uma oportunidade para explorar a aplicação prática de conceitos teóricos, destacando a importância do equilíbrio entre eficiência, robustez e clareza no desenvolvimento de algoritmos.

2 Algoritmos Elementares

2.1 Função findMinDifference

Sejam:

- n o tamanho de `elem1`
- m o tamanho de `elem2`

O algoritmo agora percorre ambos os arrays com dois ponteiros, movendo cada um conforme as comparações:

- Inicialização dos ponteiros i e j : $\mathcal{O}(1)$
- Comparação e cálculo da diferença: $\mathcal{O}(1)$ por iteração
- Movimento dos ponteiros: ocorre no máximo $n + m$ vezes, pois cada ponteiro só é incrementado uma vez.

Portanto, a complexidade total é dada por:

$$\mathcal{O}(n + m)$$

Conclusão: A complexidade temporal da função otimizada `findMinDifference` é:

$$\boxed{\mathcal{O}(n + m)}$$

2.2 Função counter

Seja n o tamanho do array (`array.size`).

1. `if (k > array.size) return Pair(0, 0)`

Verificação condicional que ocorre uma única vez:

$$\mathcal{O}(1)$$

2. `for (i in 0 until k) sum += array[i]`

Soma dos primeiros k elementos para calcular a primeira janela:

$$\mathcal{O}(k)$$

3. `if (sum < lower) ... if (sum > upper) ...`

Duas comparações após o loop:

$$\mathcal{O}(1)$$

4. `for (i in k until array.size)`

Loop que percorre o restante do array, atualizando a soma da janela deslizante com apenas duas operações constantes (adição e subtração):

$$n - k \text{ iterações} \Rightarrow \mathcal{O}(n - k)$$

Cada iteração faz:

- `sum += array[i]` e `sum -= array[i - k]`: $2 \times \mathcal{O}(1)$
- Duas comparações com `lower` e `upper`: $2 \times \mathcal{O}(1)$

Logo, este loop tem complexidade:

$$\mathcal{O}(n - k)$$

Somando os termos dominantes:

$$\mathcal{O}(1) + \mathcal{O}(k) + \mathcal{O}(n - k) = \mathcal{O}(n)$$

Conclusão: A complexidade temporal do algoritmo `counter` é:

$$\boxed{\mathcal{O}(n)}$$

2.3 Função `createSortedPartitions`

Seja:

- N o número total de linhas do ficheiro (número de inteiros a processar)
- K o tamanho de cada partição (`partitionSize`)

Loop Principal

O loop `while (true)` percorre todas as N linhas do ficheiro, lendo uma linha por iteração:

$$\text{Leitura de linhas: } \mathcal{O}(N)$$

Cada linha lida é inserida numa estrutura de heap com custo:

Inserção na heap: $\mathcal{O}(\log K)$

Portanto, o custo total das inserções é:

$$\mathcal{O}(N \log K)$$

Escrita das Partições Ordenadas

Para cada partição (de tamanho até K), chamamos a função `createSortFile`, que remove os elementos da heap um por um (isto é, realiza um heap sort):

Remoção de K elementos da heap: $K \cdot \mathcal{O}(\log K) = \mathcal{O}(K \log K)$

Como existem aproximadamente $\lceil N/K \rceil$ partições, o custo total da ordenação e escrita será:

$$\left\lceil \frac{N}{K} \right\rceil \cdot \mathcal{O}(K \log K) = \mathcal{O}(N \log K)$$

Complexidade Total

Somando as partes:

$$\mathcal{O}(N) \text{ (leitura)} + \mathcal{O}(N \log K) \text{ (inserção + ordenação)} = \mathcal{O}(N \log K)$$

Conclusão: A complexidade temporal da função `createSortedPartitions` é:

$$\boxed{\mathcal{O}(N \log K)}$$

3 Análise e Desempenho

3.1 Função `xpto`

Neste tópico, foi-nos mostrado o seguinte código em Kotlin:

```
fun xpto(v: IntArray, n: Int) {
    if (n <= 1)
        return
    var n1 = n
    n1 = n1 / 2
    for (i in 0 ..< n1)
        v[i] = v[2*i] + v[2*i + 1]
    xpto(v, n1)
```

}

Código 1: Função `xpto` em Kotlin

Analisando esta função `xpto` percebemos:

- Se $n \leq 1$, a função `xpto` simplesmente retorna;
- Caso contrário, o valor de `n1` é atualizado para metade deste ($n/2$), ou seja, o tamanho do novo subproblema é reduzido pela metade;
- O `for loop` percorre `n1` elementos, ou seja, $n/2$ elementos, e realiza uma operação de soma e atribuição de um valor a `v`.
- A função `xpto` é chamada recursivamente, com `n1`, i.e., com metade ($n/2$) do tamanho original.

Seja $T(n)$ o tempo de execução do algoritmo em questão para um *array* de tamanho n . O tempo de execução pode ser determinado por:

- O `for` que percorre $n/2$ elementos e realiza uma operação de soma, constante $[O(1)]$, em cada um. Assim, o tempo de execução deste `for` é de $O(n/2) = O(n)$.
- A chamada recursiva ocorre sobre um subproblema de tamanho $n/2$.

Com isto, a relação de recorrência do tempo de execução é a seguinte:

$$T(n) = T\left(\frac{n}{2}\right) + O(n), \quad (1)$$

ou seja, o tempo necessário para processar n elementos é igual ao tempo para processar $n/2$ elementos, somado ao tempo gasto no laço `for` (que é $O(n)$).

Resolvendo a recorrência:

$$T(n) = T\left(\frac{n}{2}\right) + O(n) \quad (2)$$

Fazendo a mudança de variável, $n \rightarrow \frac{n}{2}$:

$$T\left(\frac{n}{2}\right) = T\left(\frac{n}{4}\right) + O\left(\frac{n}{2}\right) \quad (3)$$

Substituindo (3) na (2):

$$T(n) = [T\left(\frac{n}{4}\right) + O\left(\frac{n}{2}\right)] + O(n) = T\left(\frac{n}{4}\right) + O(n) + O\left(\frac{n}{2}\right) \quad (4)$$

Novamente, fazendo a mudança de variável, $n \rightarrow \frac{n}{2}$ na (3):

$$T\left(\frac{n}{4}\right) = T\left(\frac{n}{8}\right) + \mathcal{O}\left(\frac{n}{4}\right) \quad (5)$$

Substituindo (5) na (4):

$$T(n) = [T\left(\frac{n}{8}\right) + \mathcal{O}\left(\frac{n}{4}\right)] + \mathcal{O}(n) + \mathcal{O}\left(\frac{n}{2}\right) = T\left(\frac{n}{8}\right) + \mathcal{O}(n) + \mathcal{O}\left(\frac{n}{2}\right) + \mathcal{O}\left(\frac{n}{4}\right) \quad (6)$$

Continuando a expandir até que o tamanho do subproblema seja $n = 1$, teremos:

$$T(n) = \mathcal{O}(n) + \mathcal{O}\left(\frac{n}{2}\right) + \mathcal{O}\left(\frac{n}{4}\right) + \dots + \mathcal{O}(1) \quad (7)$$

O número de passos que decorre no processo de redução do tamanho n para 1 ocorre em passos onde n é dividido por 2 a cada etapa.

O número total de passos é obtido por:

$$\frac{n}{2^k} = 1 \Leftrightarrow n = 2^k \Leftrightarrow \log_2 n = \log_2 2^k \Leftrightarrow k = \log_2 n \quad (8)$$

Logo, a soma da série tem $\log_2 n$ termos.

A série geométrica (7) é limitada por:

$$\mathcal{O}(n) \sum_{i=0}^{\log_2 n} \frac{1}{2^i} \quad (9)$$

A soma dessa série geométrica é limitada por 2:

$$\mathcal{O}(n) \cdot 2 = \mathcal{O}(n) \quad (10)$$

Portanto, $T(n) = \mathcal{O}(n)$.

O algoritmo **xpto** tem complexidade $\mathcal{O}(n)$, isto significa, que o tempo de execução cresce linearmente com n .

A complexidade espacial deste algoritmo é dada por:

$$\mathcal{O}(n) + \mathcal{O}(\log_2 n) = \mathcal{O}(n) \quad (11)$$

A parcela ($\mathcal{O}(n)$) diz respeito ao espaço do *array*, já a parcela ($\mathcal{O}(\log_2 n)$) corresponde à recursividade usada neste algoritmo. O $\mathcal{O}(\log_2 n)$ não altera a complexidade espacial final, que é $\mathcal{O}(n)$.

3.2 Função someSort

Para o segundo exercício desta secção 2 da série de exercícios, foi-nos apresentado o seguinte código em Kotlin:

```
fun someSort(array: IntArray, l: Int, r: Int) {  
    if (r <= l) return  
    if (r == l + 1) {  
        if (array[l] > array[r])  
            exchange(array, l, r)  
    } else {  
        val p = (r - l + 1) / 3  
        someSort(array, l, r - p)  
        someSort(array, l + p, r)  
        someSort(array, l, r - p)  
    }  
}
```

Código 2: Função someSort em Kotlin

Analisando esta função `someSort` percebemos:

- Se $r \leq l$, o `array` é vazio ou tem apenas um elemento, então a função retorna imediatamente.
- Se $r == l + 1$, o `array` tem exatamente 2 elementos (`array[l]` e `array[r]`), eles são comparados e trocados, caso `array[l] > array[r]`.
- No caso onde $p = r > l + 1$, o tamanho do `array` é $n = r - l + 1$. Calcula $p = \frac{(r-l+1)}{3}$, ou seja, divide o `array` em 3 partes, aproximadamente. De seguida faz 3 chamadas recursivas: `someSort (array, l, r-p)`, `someSort (array, l+p, r)` e `someSort (array, l, r-p)`.

O objetivo do algoritmo é organizar os elementos do `array` de forma ordenada, ao longo das chamadas recursivas.

Seja $T(n)$ o tempo de execução do algoritmo para um `subarray` de tamanho n .

- Se $n \leq 1$, $T(n) = \mathcal{O}(1)$, pois há apenas um elemento ou nenhum.
- Se $n = 2$, há no máximo uma comparação e uma troca, o que é $\mathcal{O}(1)$.
- Para $n > 2$, há três chamadas recursivas, cada uma delas operando sobre `subarrays` de tamanho, aproximadamente, $\frac{2}{3}n$, (pois $p = \frac{n}{3}$ e $r - p$ cobre $\frac{2}{3}n$ elementos).

A recorrência fica:

$$T(n) = 3T\left(\frac{2n}{3}\right) + \mathcal{O}(1) \quad (12)$$

Segue-se a resolução da recorrência (12):

$$T(n) == 3T\left(\frac{2n}{3}\right) + \mathcal{O}(1) \quad (13)$$

Fazendo a mudança de variável, $n \rightarrow \frac{2n}{3}$, temos:

$$T\left(\frac{2n}{3}\right) = 3T\left(\frac{4n}{9}\right) + \mathcal{O}(1) \quad (14)$$

Substituindo (14) na (13):

$$T(n) = 3[3T\left(\frac{4n}{9}\right) + \mathcal{O}(1)] + \mathcal{O}(1) = 9T\left(\frac{4n}{9}\right) + 3\mathcal{O}(1) + \mathcal{O}(1) \quad (15)$$

Após k expansões:

$$T(n) = 3^k T\left(\frac{2^k}{3^k} n\right) + \mathcal{O}(3^k) \quad (16)$$

A recorrência para quando $\frac{2^k}{3^k} n \leq 1$, ou seja, $2^k n \leq 3^k$. Resolvendo, através de cálculos auxiliares:

$$2^k n = 3^k \quad (17)$$

$$\Leftrightarrow \log(2^k n) = \log 3^k \quad (18)$$

$$\Leftrightarrow \log 2^k \cdot \log n = \log 3^k \quad (19)$$

$$\Leftrightarrow \log n = \frac{\log 3^k}{\log 2^k} \quad (20)$$

$$\Leftrightarrow \log n = k \cdot \log\left(\frac{3}{2}\right) \quad (21)$$

$$\Leftrightarrow k = \frac{\log n}{\log\left(\frac{3}{2}\right)} = \log_{\frac{3}{2}} n \quad (22)$$

Como o termo $(\mathcal{O}(3^k))$ é o dominante, temos:

$$T(n) = \mathcal{O}(3^{\log_{\frac{3}{2}} n}) \quad (23)$$

Aplicando uma mudança de base, $3^{\log_{\frac{3}{2}} n} = n^{\log_{\frac{3}{2}} 3}$, obtemos:

$$T(n) = \mathcal{O}(n^{\log_{\frac{3}{2}} 3}) \approx \mathcal{O}(n^{2.71}) = \mathcal{O}(n^3) \quad (24)$$

Concluindo, o tempo de execução deste algoritmo `someSort` é $\mathcal{O}(n^3)$, que é mais lento que a complexidade $\mathcal{O}(n \log n)$ e $\mathcal{O}(n^2)$. Apesar do valor exato ser de uma exponencial inferior a 3, a complexidade temporal deste algoritmo é de $\mathcal{O}(n^3)$, que representa o menor dos limites superiores que delimitam por completo a função. Esta complexidade não é desejada para algoritmos pelo facto de ser muitíssimo lenta para qualquer tamanho de *array*, e quanto maior for o tamanho do *array*, mais lento será.

3.3 Proposta de algoritmo de ordenação

Para este exercício, que pede para implementar um algoritmo de ordenação estável que seleciona o i -ésimo menor elemento em cada iteração, propomos uma implementação baseada no *Selection Sort*, mas estável.

O *Selection Sort* convencional não é estável porque troca elementos não adjacentes (consecutivos). Para torná-lo estável:

- Em vez de trocar elementos, deslocamos os elementos para manter a ordem relativa dos elementos iguais (de valor igual);
- Identificamos o mínimo na parte não ordenada do *array*;
- Movemos esse elemento para sua posição correta, deslocando os elementos intermédios.

3.3.1 Implementação proposta

A implementação, proposta por nós, segue-se a diante:

```
fun stableSelectionSort(arr: IntArray) {  
    val n = arr.size  
    for (i in 0 until n - 1) {  
        var minIdx = i  
        for (j in i + 1 until n) {  
            if (arr[j] < arr[minIdx]) {  
                minIdx = j  
            }  
        }  
  
        if (minIdx != i) {  
            val minValue = arr[minIdx]  
            for (k in minIdx downTo i + 1) {  
                arr[k] = arr[k - 1]  
            }  
            arr[i] = minValue  
        }  
    }  
}
```

Código 3: Função `stableSelectionSort` em Kotlin

A estabilidade deste código é garantida uma vez que quando encontramos o mínimo, movemos todos os elementos entre a posição atual e a posição do mínimo para a direita. Isso também preserva a ordem original dos elementos iguais, para evitar trocas redundantes. Consequentemente, elementos com valores iguais ao mínimo não "saltam" à frente de outros elementos iguais que apareciam antes no *array* original.

Para este algoritmo apresentado por nós, em seguida, demonstramos a análise formal em notação Big Oh.

3.3.2 Complexidade Temporal

O `for loop` inicial executa $\mathcal{O}(n)$ iterações ($n - 1$ iterações, exatamente) (de $i = 0$ até $n - 2$)

No `loop` interno, quando $i = 0$, i.e., no pior caso, executa $n - 1$ iterações. Já no melhor caso (quando $i = n - 2$), executa apenas 1 iteração.

Sendo assim, o número total de comparações dá-se por:

$$\sum_{i=0}^{n-2} (n - i - 1) = \frac{n(n-1)}{2} \rightarrow \mathcal{O}(n^2) \quad (25)$$

Já para o terceiro `for` do código, no pior caso, quando o mínimo está na última posição, para $i = 0$ executa $n - 1$ deslocamentos. Para $i = n - 2$, executa 1 deslocamento.

Novamente, e semelhante ao anterior, o número total de deslocamentos é dado por:

$$\sum_{i=0}^{n-2} (n - i - 1) = \frac{n(n-1)}{2} \rightarrow \mathcal{O}(n^2) \quad (26)$$

Em suma, tanto no pior como no melhor caso, o algoritmo apresenta uma complexidade temporal de $\mathcal{O}(n^2)$.

3.3.3 Complexidade Espacial

Neste algoritmo apenas são usadas variáveis temporárias (`i`, `j`, `k`, `minIdx`, `minValue`), não há nenhuma estrutura de dados adicional criada e apenas temos o espaço auxiliar ($\mathcal{O}(1)$), que é de complexidade espacial constante.

Comparando este código com o *Selection Sort*, visto que a nossa implementação é muito baseada neste algoritmo, porém modificada para ser estável para satisfazer o objetivo da questão, percebemos as diferenças/ semelhanças e vantagens/ desvantagens de cada um em específico:

- O nosso código, baseado no *Selection Sort*:

Preserva a ordem relativa de elementos iguais ao mover blocos de elementos. Porém, não realiza trocas diretas entre elementos não adjacentes.

- *Selection Sort* original:

O verdadeiro *Selection Sort*, que é instável, seria implementado com simples trocas, e este teria a mesma complexidade temporal de $\mathcal{O}(n^2)$, menos operações de movimentação (apenas $\mathcal{O}(n)$ trocas no total) e não manteria a estabilidade.

A escolha de um *Selection Sort* modificado foi adequada porque mantém a propriedade de selecionar o i -ésimo menor elemento em cada iteração, é simples de implementar e preserva a ordenação *in-place* (sem alocação adicional significativa).

Concluindo, o nosso código possui complexidade temporal de $\mathcal{O}(n^2)$ em todos os casos e uma complexidade espacial de $\mathcal{O}(1)$.